UNIVERSIDAD GALILEO

P D D S

CURSO: CIBERSEGURIDAD

# Laboratorio: SAST local y pipeline CI con Bearer CLI

**Pablo Agreda**

# Resumen del trabajo

El laboratorio consistió en aplicar un análisis estático de seguridad (SAST) sobre el código de OWASP Juice Shop con Bearer CLI, interpretar los hallazgos y automatizar el proceso en un pipeline de GitHub Actions. También se construyó una demostración controlada: se agregó una ruta vulnerable en una rama, se abrió un Pull Request, se comprobó que el pipeline marcó las alertas y, después de corregir el código, se verificó que las alertas se cerraron.

El repositorio del proyecto es público y contiene el código de Juice Shop y el workflow `sast-bearer.yml`. El análisis local encontró 397 hallazgos (21 críticos, 22 altos, 32 medios y 322 bajos) y el pipeline publicó los resultados en *Security → Code scanning*, además de generar un reporte HTML descargable como artefacto de la ejecución.

## Ficha del análisis

| Dato | Detalle |
|---|---|
| Herramienta | Bearer CLI 2.1.2, ejecutada con Docker (`bearer/bearer:latest-amd64`). |
| Objetivo | OWASP Juice Shop, código fuente completo (rama `main`). |
| Formatos generados | HTML para lectura y SARIF para GitHub Code scanning. |
| Resultado local | 397 hallazgos: 21 críticos, 22 altos, 32 medios y 322 bajos. |
| Automatización | GitHub Actions en push, Pull Request y ejecución manual. |
| Demostración | 2 alertas críticas detectadas en un PR y cerradas después de la corrección. |

## Enlaces del proyecto

| Recurso | Enlace |
|---|---|
| Repositorio | github.com/pagreda888/juice-shop-sast |
| Workflow | .github/workflows/sast-bearer.yml |
| Pull Request | Pull Request #1 — Demostración SAST |
| Commit base | `7b8ab1a` — Archivos base de Juice Shop |
| Commit del workflow | `0570977` — Add Bearer SAST workflow |
| Commit vulnerable | `e272eb6` — Se agrega ruta vulnerable para demostración SAST |
| Commit corregido | `eb85cd2` — Se corrige la ruta de demostración |

## Contenido


1. Parte A — Repositorio · preparación
2. Parte B — Análisis local · hallazgos
3. Parte C — Pipeline de CI · GitHub Actions
4. Parte D — Demostración en el PR · detección y corrección
5. Parte E — Preguntas de análisis
6. Conclusiones

# 1. Parte A — Repositorio de trabajo

Se clonó OWASP Juice Shop desde su repositorio oficial y se eliminaron la carpeta `.git` y la carpeta `.github/workflows`, con el fin de quedarse únicamente con el código de la aplicación y construir un repositorio propio. También se eliminaron archivos de configuración de herramientas de desarrollo que no forman parte de la aplicación.

Después se inicializó un repositorio nuevo con rama `main`, se agregó el remoto y se hizo el primer commit con el mensaje «Archivos base de Juice Shop» (`7b8ab1a`). El repositorio quedó público con el nombre **juice-shop-sast**.

En el archivo `.gitignore` se agregaron las reglas `*.sarif` y `reports/` para que los reportes locales no se suban al repositorio. Eso mantiene limpio el historial y evita publicar archivos que se regeneran en cada análisis.

> **Detalle importante:** Bearer solo analiza archivos que git tiene registrados. Al principio el escaneo devolvía cero hallazgos porque el repositorio todavía no tenía commit; después del primer commit el análisis encontró el código normalmente. En el pipeline esto no ocurre porque el checkout descarga el repositorio ya versionado.

### Preparación del repositorio

Los comandos principales de esta etapa fueron los siguientes. El `--force` del push se usó una sola vez porque GitHub había creado un `README.md` automático al dar de alta el repositorio y las dos historias no estaban relacionadas; el push reemplazó ese archivo inicial por el código de la aplicación.

```
git clone https://github.com/juice-shop/juice-shop.git juice-shop-sast
cd juice-shop-sast
rm -rf .git .github/workflows
git init -b main
git add -A
git commit -m "Archivos base de Juice Shop"
git remote add origin https://github.com/pagreda888/juice-shop-sast.git
git push -u --force origin main
```

Las reglas agregadas al archivo `.gitignore` fueron:

```
# Reportes locales de SAST
*.sarif
reports/
```

Con eso, los reportes que se generan en cada análisis local quedan fuera del control de versiones y el repositorio conserva únicamente el código de la aplicación y el workflow del pipeline.

## 2. Parte B — Análisis local con Bearer CLI

Bearer se ejecutó localmente con Docker, usando la imagen `bearer/bearer:latest-amd64`. El contenedor monta la carpeta del proyecto en `/tmp/scan` y escribe los reportes en la carpeta `reports/`. El comando base fue el siguiente:

```
docker run --rm -v "$PWD:/tmp/scan" bearer/bearer:latest-amd64 scan /tmp/scan \
  --format html --output /tmp/scan/reports/bearer-report.html --exit-code 0
```

Para el reporte SARIF se cambió `--format sarif`. Para filtrar únicamente los hallazgos críticos y altos se usó `--severity critical,high`, y para excluir carpetas que no son código de la aplicación se agregó `--skip-path "test/*,cypress/*"`. Bearer ya omite archivos de prueba por defecto, pero se dejó explícita la exclusión para el ejercicio.

El escaneo completo revisó 175 reglas y encontró **397 hallazgos**: 21 críticos, 22 altos, 32 medios y 322 bajos. Los reportes generados fueron `bearer-report.html`, `bearer-report.sarif` y `bearer-critical-high.html`. El reporte HTML se adjunta a esta entrega y también se copió dentro de la carpeta `entregables/`.

```
git push
0x1cedcafe@MacBook-Pro-de-Pablo juice-shop-sast % docker run --rm -v "$PWD:/tmp/scan" bearer/bearer:latest-amd64 scan /tmp/scan --disable-version-check | tail -25
WARNING: The requested image's platform (linux/amd64) does not match the detected host platform (linux/arm64/v8) and no specific platform was requested
Analyzing codebase
⠏  (173904/-) [0s]
Loading rules
Scanning target /tmp/scan
 └ 100% [==============] (1404/1404) [32s]
Running Detectors
Generating dataflow
Evaluating rules
 └ 100% [==============] (471/471) [2s]
LOW: Leakage of information in logger message [CWE-532]
https://docs.bearer.com/reference/rules/javascript_lang_logger_leak
To ignore this finding, run: bearer ignore add 602478eb7d16a5a0e5086caf562308ec_5

File: /tmp/scan/server.ts:774

 774     logger.info(colors.cyan(`Server listening on port ${colors.bold(`${port}`)}`))

LOW: Leakage of information in logger message [CWE-532]
https://docs.bearer.com/reference/rules/javascript_lang_logger_leak
To ignore this finding, run: bearer ignore add 602478eb7d16a5a0e5086caf562308ec_6

File: /tmp/scan/server.ts:777

 777       logger.info(colors.cyan(`Server using proxy base path ${colors.bold(`${process.env.BASE_PATH}`)} for redirects`))
========================================

175 checks, 397 findings

CRITICAL: 21 (CWE-502, CWE-798, CWE-89, CWE-95)
HIGH: 22 (CWE-22, CWE-312, CWE-73, CWE-79, CWE-918)
MEDIUM: 32 (CWE-1333, CWE-208, CWE-328, CWE-532, CWE-601, CWE-693)
LOW: 322 (CWE-330, CWE-532, CWE-548)
WARNING: 0
```

**Figura 1.** Ejecución local de Bearer con Docker. Al final se observa el resumen: 175 verificaciones y 397 hallazgos, con el desglose por severidad.

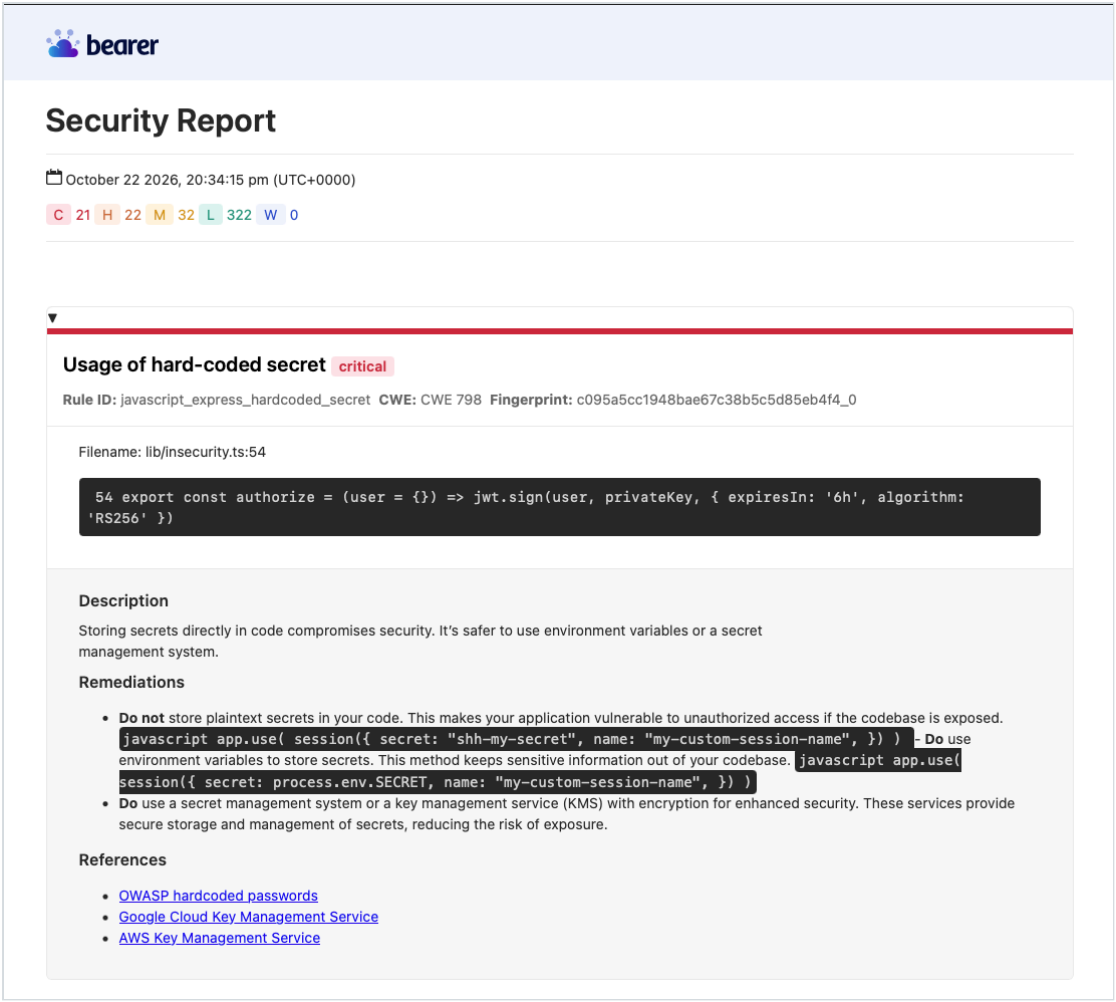



**Figura 2.** Reporte HTML abierto en el navegador. Se muestra uno de los hallazgos críticos: secreto embebido en `lib/insecurity.ts:54`.

### Hallazgos más importantes

De los 397 resultados se seleccionaron los cinco que, por severidad y por el tipo de vulnerabilidad, representan el mayor riesgo para la aplicación:

| # | Regla / ID | Severidad | CWE | Archivo:línea | ¿Verdadero o falso positivo? ¿Por qué? |
|---|---|---|---|---|---|
| 1 | `javascript_lang_eval_user_input` | CRÍTICA | 95 | `routes/b2bOrder.ts:22` | **Verdadero positivo.** `body.orderLinesData` llega a `vm.runInContext('safeEval(...)')`. El sandbox de `notevil` puede escaparse, por eso existe el reto de ejecución remota de código. |
| 2 | `javascript_lang_sql_injection` | CRÍTICA | 89 | `routes/login.ts:34` | **Verdadero positivo.** El correo y el hash de la contraseña se concatenan directamente en la consulta SQL; permite modificar la condición y saltarse el inicio de sesión. |
| 3 | `javascript_lang_sql_injection` | CRÍTICA | 89 | `routes/search.ts:23` | **Verdadero positivo.** `req.query.q` se interpola dentro de un `LIKE`; el parámetro permite una inyección SQL por `UNION`. |
| 4 | `javascript_lang_jwt_hardcoded_secret` | CRÍTICA | 798 | `lib/insecurity.ts:54` | **Verdadero positivo.** La llave privada RSA está embebida en el código (línea 21) y se usa para firmar los JWT. Cualquiera con acceso al código puede forjar tokens. |
| 5 | `javascript_express_server_side_request_forgery` | ALTA | 918 | `routes/profileImageUrlUpload.ts:24` | **Verdadero positivo.** Se hace `fetch(url)` con `req.body.imageUrl` sin validar el destino; permite solicitudes del servidor hacia recursos internos (SSRF). |

> El reporte filtrado por `critical,high` y sin las carpetas `test/` y `cypress/` se guardó como `bearer-critical-high.html`. El reporte completo es el que se entrega en `entregables/bearer-report.html`.

## 3. Parte C — Pipeline de integración continua

El pipeline se construyó en el archivo `.github/workflows/sast-bearer.yml`. Se ejecuta en cada push a `main`, en cada Pull Request hacia `main` y también de forma manual mediante `workflow_dispatch`. Los permisos se limitaron a `contents: read` y `security-events: write`, que son los necesarios para descargar el código y publicar los resultados de Code scanning.

El trabajo descarga el repositorio con `actions/checkout`, instala Bearer con el instalador oficial y ejecuta dos análisis: uno en formato SARIF y otro en formato HTML. El SARIF se sube a *Security → Code scanning* con `github/codeql-action/upload-sarif` y el HTML se publica como artefacto con `actions/upload-artifact`. Los dos pasos de carga usan `if: always()` para que se ejecuten aunque el análisis encuentre vulnerabilidades.

### Cómo se cumple cada requisito

| # | Requisito | Implementación |
|---|---|---|
| 1 | Push a main, Pull Request y ejecución manual | Bloques `on.push`, `on.pull_request` y `on.workflow_dispatch`. |
| 2 | Permisos mínimos | `contents: read` y `security-events: write`. |
| 3 | Descargar el código y ejecutar Bearer | `actions/checkout@v7` y `bearer scan .` sobre todo el repositorio. |
| 4 | SARIF a Security → Code scanning | `--format sarif` y `github/codeql-action/upload-sarif@v4`. |
| 5 | HTML como artefacto descargable | `--format html` y `actions/upload-artifact@v7` con el nombre `bearer-html-report`. |
| 6 | Subir resultados aunque haya hallazgos | Los análisis usan `--exit-code 0` y las cargas usan `if: always()`. |
| 7 | Cada línea del YAML comentada | El archivo incluye un comentario explicativo en cada línea. |

Se usó `--exit-code 0` de forma intencional: Bearer devuelve un código distinto de cero cuando encuentra hallazgos, y si eso ocurriera antes de subir los reportes, la carga se saltaría. Con el código 0 el pipeline siempre genera y publica los resultados; el bloqueo del merge se deja en manos del check de Code scanning, que sí marca el PR como fallido cuando aparecen alertas nuevas. Si se quisiera que el propio paso de Bearer fallara, bastaría con quitar `--exit-code 0` y agregar `--fail-on-severity critical,high`.

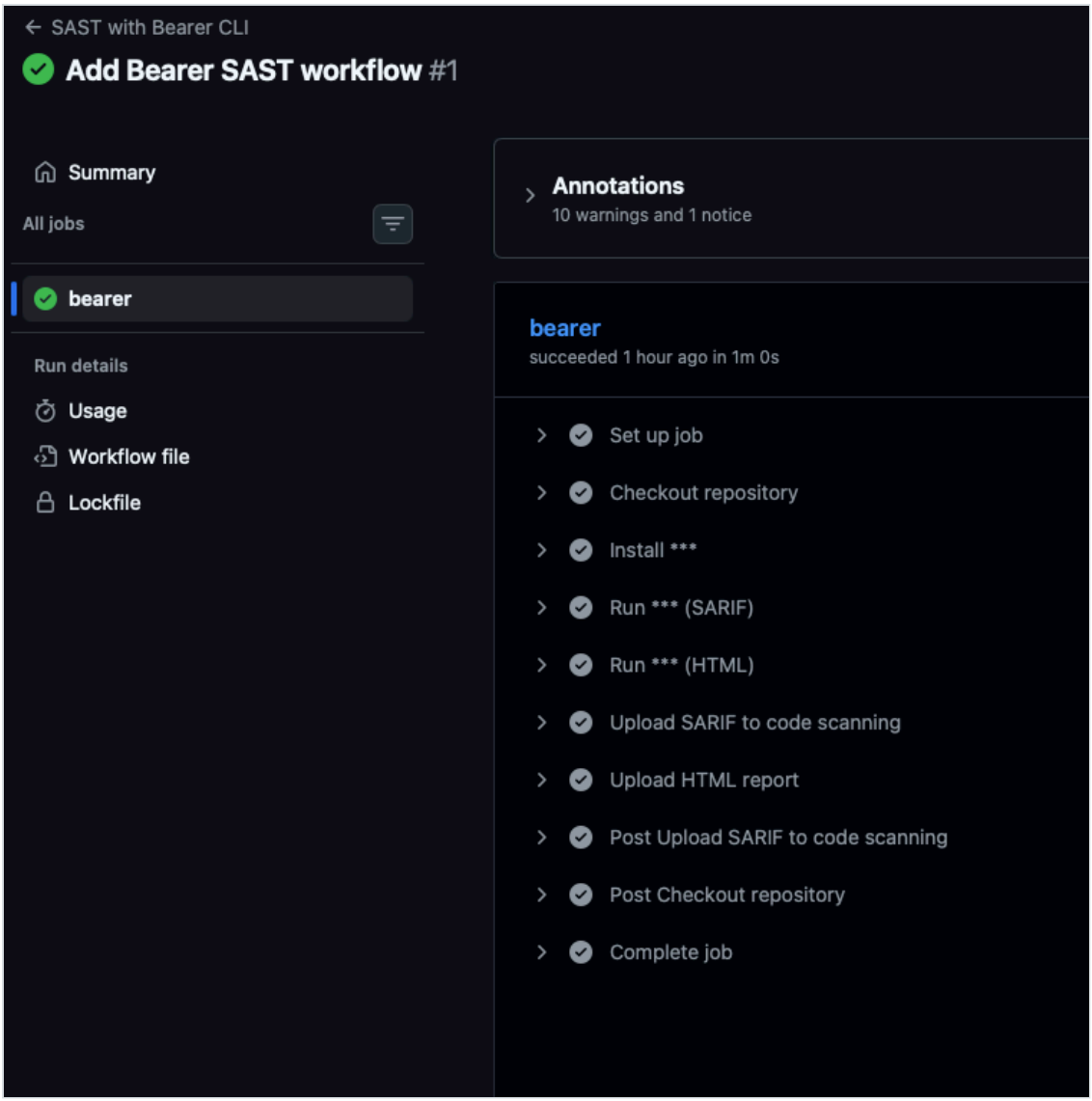



**Figura 3.** Detalle de la ejecución: todos los pasos terminaron correctamente, incluidos la carga del SARIF y la carga del reporte HTML.

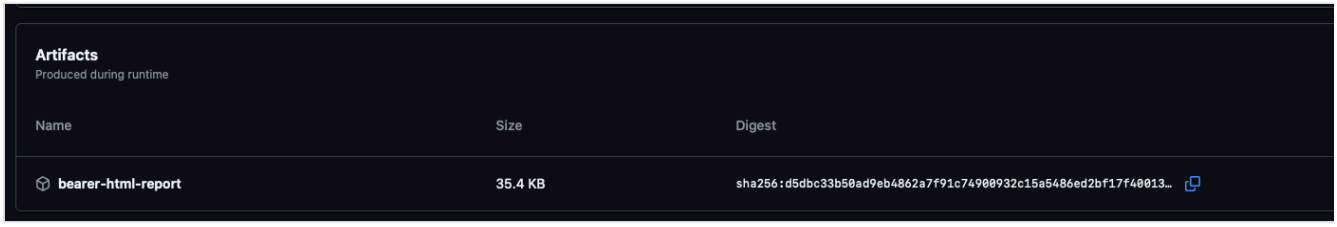



**Figura 4.** Sección de artefactos de la ejecución. El reporte HTML queda disponible para descargar como `bearer-html-report`.

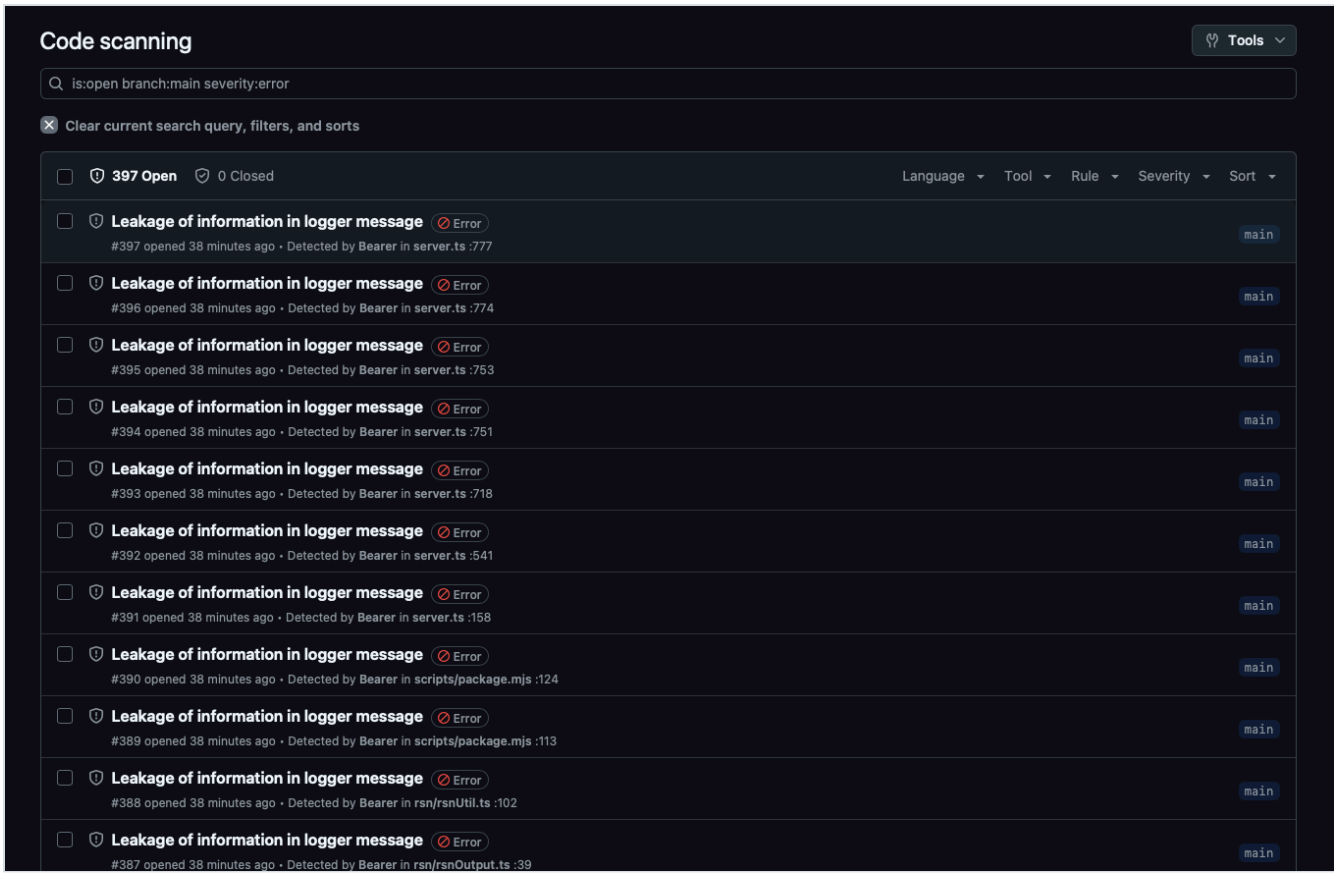



**Figura 5.** Resultados publicados en *Security → Code scanning*. GitHub registró 397 alertas abiertas detectadas por Bearer.

Esas 397 alertas corresponden a la línea base del proyecto y no se corrigen con este laboratorio. Lo importante es que el pipeline las deja visibles y clasificadas por herramienta, regla y severidad, de modo que cualquier cambio nuevo se pueda comparar contra esa base. En la Parte D se ve el caso contrario: dos alertas nuevas asociadas a un solo archivo hicieron fallar el chequeo del Pull Request, y desaparecieron cuando el código se corrigió.

## Resumen de la ejecución

| Indicador | Resultado |
|---|---|
| Ejecución | #1, commit `0570977`, duración 1 m 5 s, estado verde. |
| Pasos | Checkout, instalación, análisis SARIF, análisis HTML, carga de SARIF, carga de HTML y post-acciones. |
| Artefacto | `bearer-html-report`, 35.4 KB. |
| Alertas publicadas | 397 alertas abiertas en *Security → Code scanning*. |
| Resultado | Los dos reportes se generaron y se publicaron sin errores. |

## 4. Parte D — Demostración: cambio, detección y corrección

Para demostrar el ciclo completo se creó la rama `demo-vulnerabilidad` y se agregó el archivo `routes/demoVulnerable.ts` con dos vulnerabilidades intencionales: el uso de `eval()` sobre un parámetro de la petición y una consulta SQL construida por concatenación. El commit de esta versión es `e272eb6`.

```
const expression = String(req.query.expression ?? '')
const result = eval(expression)
models.sequelize.query(`SELECT * FROM Products WHERE name = '${expression}'`)
```

Antes de subir el cambio se ejecutó Bearer localmente sobre ese archivo. El resultado fueron dos hallazgos críticos: `javascript_lang_eval_user_input` (CWE-95) en la línea 8 y `javascript_lang_sql_injection` (CWE-89) en la línea 9. Eso confirmó que la vulnerabilidad era detectable antes de abrir el Pull Request.




**Figura 6.** Detección local del archivo de demostración: 87 verificaciones y 2 hallazgos críticos (eval y SQL injection).

Después se subió la rama y se abrió el Pull Request #1 hacia `main`. El workflow corrió automáticamente y el check *Code scanning results / Bearer* falló con el mensaje «2 new alerts including 2 errors». En la pestaña *Files changed* se ven las dos anotaciones sobre las líneas vulnerables.

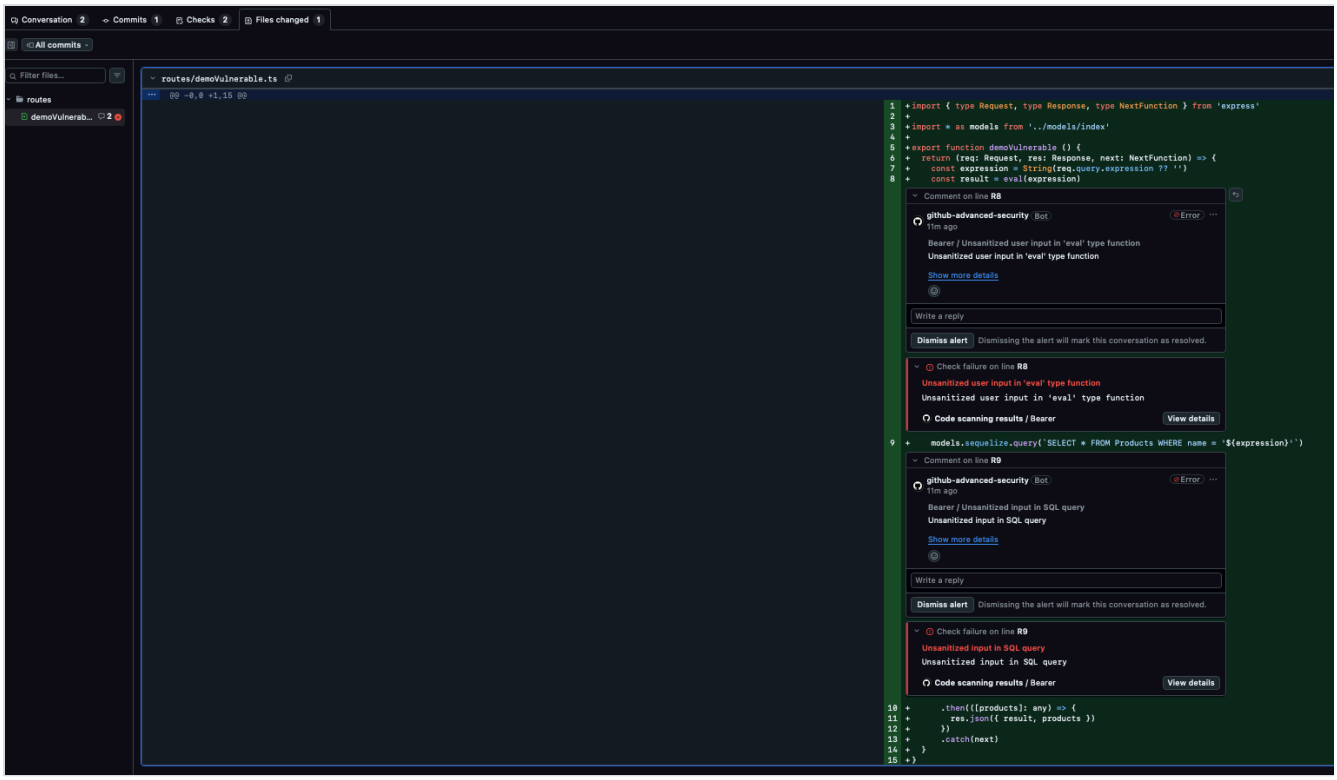



**Figura 7.** Pull Request con las dos alertas de Bearer sobre `routes/demoVulnerable.ts`: línea 8 (eval) y línea 9 (SQL injection).

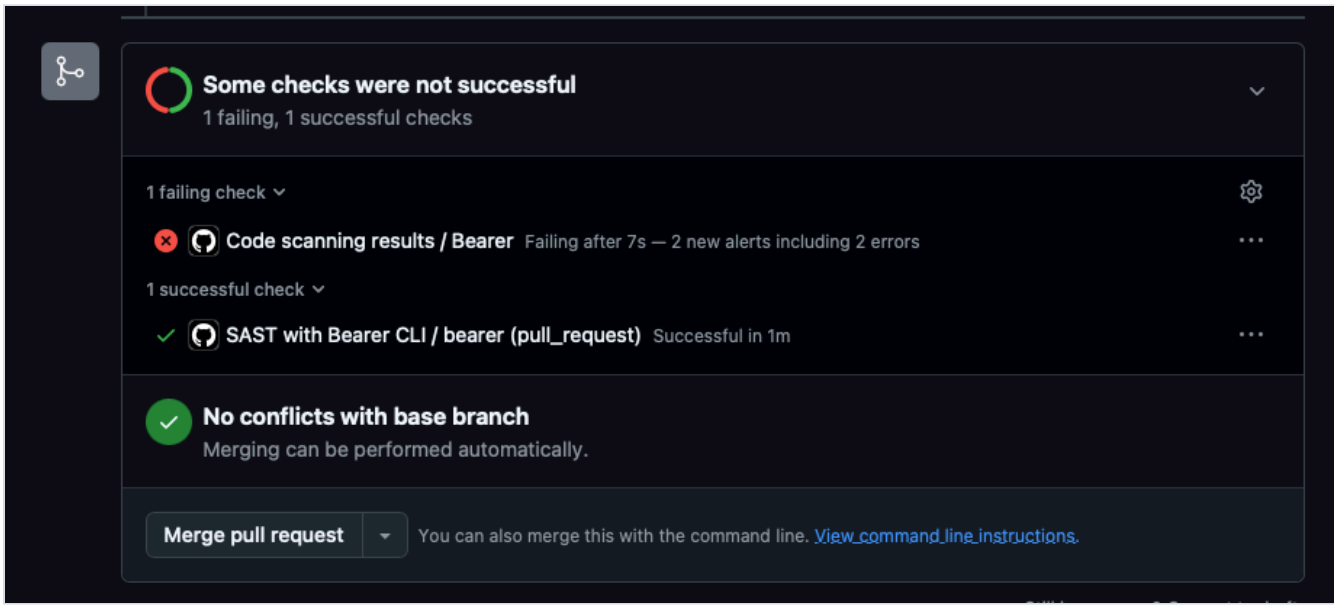



**Figura 8.** El PR queda bloqueado: el check de Code scanning reporta 2 alertas nuevas, aunque el workflow de SAST se ejecutó correctamente.

La corrección se hizo en el commit `eb85cd2`. Se eliminó `eval()` y la consulta se parametrizó con `replacements`, de modo que la entrada del usuario ya no se interpreta como código ni como parte de la sentencia SQL:

```
const name = String(req.query.name ?? '')
models.sequelize.query('SELECT * FROM Products WHERE name = ?', { replacements: [name] })
```

Después del push, el análisis local ya no reportó hallazgos en ese archivo y el PR cambió a verde: «All checks have passed» y «No new alerts in code changed by this pull request». Las alertas quedaron

cerradas. Es importante notar que esas dos alertas nunca llegaron a `main`, porque la versión vulnerable solo existió en la rama del PR; por eso el cierre se demuestra en el chequeo del Pull Request y no en el filtro `branch:main is:closed`.

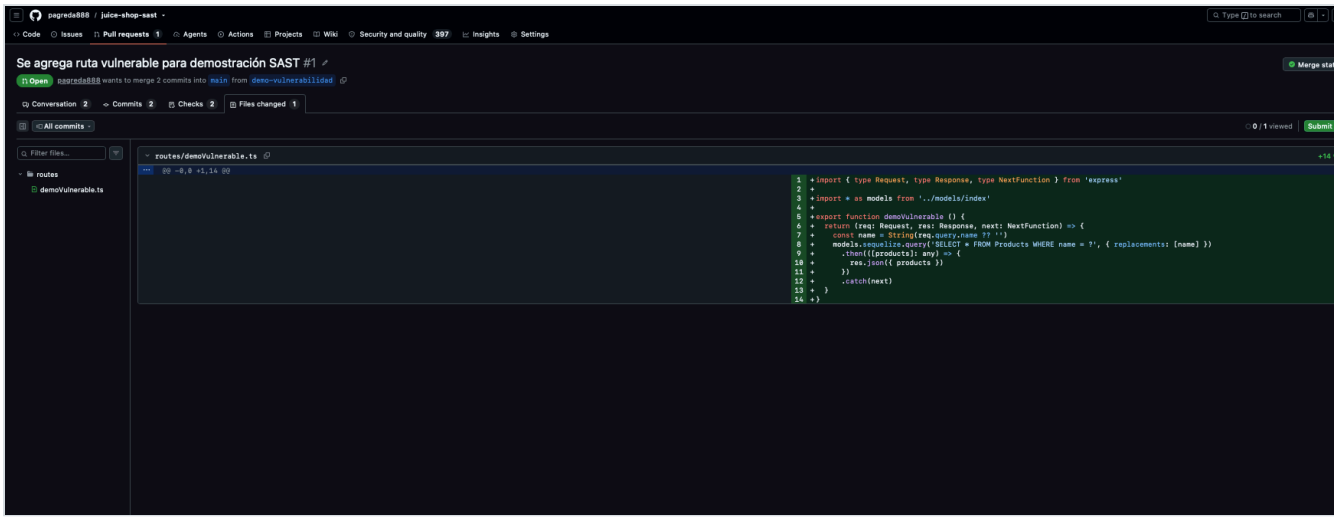



**Figura 9.** Versión final del archivo en el PR: sin `eval()` y con la consulta parametrizada.

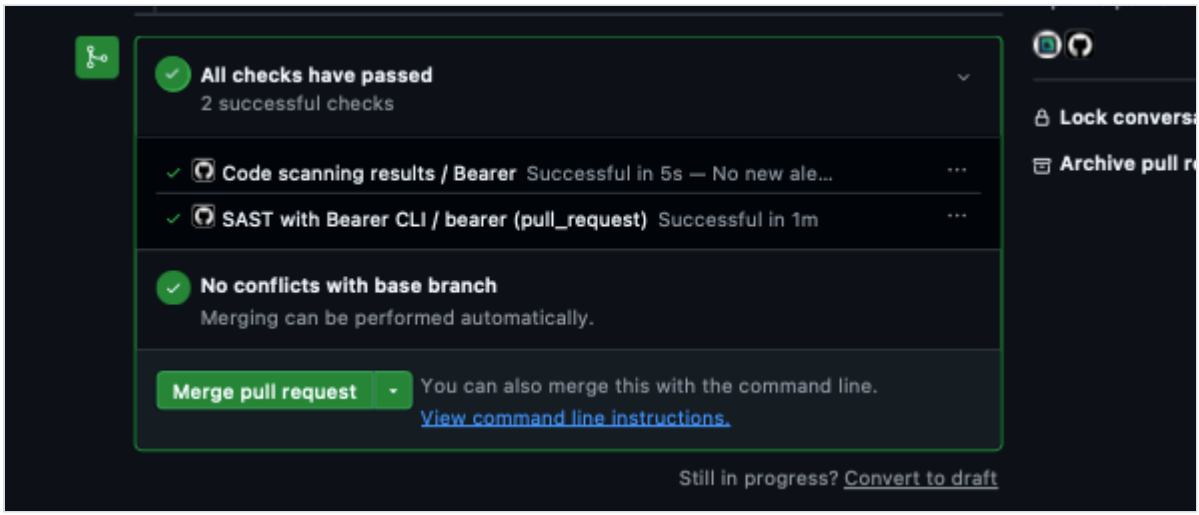



**Figura 10.** El PR pasa a verde después de la corrección: «No new alerts in code changed by this pull request».

Enlace del PR: github.com/pagreda888/juice-shop-sast/pull/1. El PR conserva los dos commits para que se vea el historial completo del cambio vulnerable y su corrección.

## 5. Parte E — Preguntas de análisis

### 14. ¿Qué diferencias encontraste entre el análisis local y el del pipeline? ¿Por qué conviene tener los dos?

Al hacer el laboratorio noté que el análisis local y el del pipeline se parecen en el resultado, pero se usan en momentos distintos. En local ejecuté Bearer con Docker y pude ver el detalle completo de cada hallazgo, repetir el escaneo las veces que quisiera y revisar el reporte HTML con calma antes de subir nada. En el pipeline, en cambio, el análisis corre solo en cada push y en cada Pull Request, en una máquina limpia, y los resultados quedan registrados en Security → Code scanning y en el artefacto HTML de la ejecución. Esa diferencia es la que más se siente: lo local sirve para trabajar y corregir, y el pipeline sirve como control y como evidencia de que el código que se integra pasó por el análisis.

También vi dos detalles que muestran por qué conviene tener los dos. El primero es que la versión de las reglas influye: con una versión nativa anterior obtuve 367 hallazgos, mientras que con la versión 2.1.2 que usa el pipeline obtuve 397. El segundo es que Bearer solo analiza archivos que git tiene registrados; al principio el escaneo local salió vacío porque el repositorio aún no tenía commit. En el pipeline eso no pasa porque el checkout ya trae el repositorio versionado. Por eso el local no reemplaza al pipeline: el local es rápido y cómodo, pero depende de que uno se acuerde de ejecutarlo; el pipeline es obligatorio y deja historial.

### 15. ¿Qué ventajas y limitaciones tiene SAST frente a DAST?

La ventaja principal de SAST es que revisa el código sin necesidad de ejecutar la aplicación. Eso permite encontrar el problema en la etapa de desarrollo, señalar el archivo y la línea exactos y cubrir caminos que quizá nunca se ejecutarían en una prueba manual. En este laboratorio eso se vio claramente: Bearer encontró el `eval` de `b2bOrder.ts`, las inyecciones SQL de `login.ts` y `search.ts` y la llave privada embebida en `insecurity.ts`, todo sin levantar Juice Shop.

La limitación es que no todo se puede ver desde el código. Juice Shop tiene fallos de autorización, IDOR, manipulación de precios y otras vulnerabilidades de lógica de negocio que un análisis estático difícilmente entiende, y además este tipo de herramientas genera falsos positivos. DAST, al probar la aplicación en ejecución, sí ve el comportamiento real, la configuración del servidor, las sesiones y la lógica de negocio, y confirma si algo es explotable. Pero llega más tarde, necesita un entorno desplegado, no siempre indica la línea del código y depende de la cobertura de las pruebas. En resumen, SAST me sirvió para encontrar y ubicar; DAST haría falta para confirmar el impacto real. Lo ideal es usar SAST durante el desarrollo y DAST antes de desplegar.

### 16. ¿Bloquearías el merge de un PR si el SAST encuentra hallazgos? ¿Con qué criterio?

Sí lo bloquearía, pero no por cualquier hallazgo. En este laboratorio el PR se bloqueó con dos errores críticos y me pareció correcto, porque eran vulnerabilidades explotables: ejecución de código con `eval` e inyección SQL. Mi criterio sería bloquear los hallazgos nuevos de severidad crítica y alta, sobre todo si son de clases como inyección, ejecución de código, bypass de autenticación, deserialización insegura o secretos embebidos. También revisaría el tipo de regla y no solo la severidad, porque un hallazgo alto en una parte del código que no se ejecuta no es lo mismo que uno en una ruta pública.

Lo que no haría es bloquear por los 397 hallazgos que ya venían en Juice Shop. Son deuda previa del proyecto y, si se exigiera cero hallazgos, ningún PR podría avanzar. En un proyecto real compararía contra

la línea base y solo exigiría que el cambio no agregue hallazgos nuevos. Los de severidad media y baja los dejaría como tarea con plazo, no como bloqueo. Las excepciones, como un falso positivo comprobado, deben quedar documentadas en `bearer.ignore` y no usarse como excusa para desactivar reglas. En el pipeline esto se puede implementar con `--fail-on-severity critical,high` y dejando la carga de reportes con `if: always()`.

### 17. Elige un hallazgo que consideres falso positivo y explica cómo lo gestionarías en Bearer sin desactivar la regla para todo el proyecto.

Elegí el hallazgo `javascript_lang_non_literal_fs_filename` en `lib/antiCheat.ts:198`, donde aparece `fs.readFileSync(path.resolve(relativePath), 'utf8')`. Lo considero un falso positivo porque `relativePath` no viene del usuario: lo utiliza internamente el detector de trampas para leer archivos de los retos, con rutas que están definidas en el propio proyecto. No hay una entrada externa que pueda manipular esa ruta, así que no hay un path traversal explotable en ese punto.

Para gestionarlo usaría el fingerprint del hallazgo y el comando `bearer ignore add`, que agrega la excepción al archivo `bearer.ignore` con autor y comentario. Así queda una excepción puntual, documentada y revisable, y la regla sigue activa para el resto del proyecto. La idea es no usar `--skip-rule`, porque desactiva la regla en todo el proyecto, ni `--skip-path`, porque oculta todo un archivo. Si el código cambia, la excepción se revisa o se elimina con `bearer ignore remove`. En el caso concreto sería algo como:

```
bearer ignore add b202373ba118027f74e94d9303f145e4_0 \
  --author "Pablo Agreda" \
  --comment "Ruta interna de archivos de retos; no proviene de entrada del usuario"
```

## 6. Conclusiones

El laboratorio permitió comprobar que Bearer CLI es una herramienta útil para encontrar problemas de seguridad directamente en el código y para integrar ese análisis en el flujo de trabajo diario. El reporte local ayudó a entender los hallazgos y a ubicarlos con archivo y línea, mientras que el pipeline convirtió ese análisis en un control automático que deja evidencia y que puede bloquear un cambio riesgoso antes de que llegue a la rama principal.

La demostración con la rama `demo-vulnerabilidad` fue la parte más clara del ejercicio: el Pull Request mostró las dos alertas sobre las líneas vulnerables y, después de corregir el código, el mismo PR pasó a verde y las alertas se cerraron. Eso confirma el objetivo del laboratorio: detectar el cambio inseguro antes de integrarlo y dejar constancia de la corrección. También quedó claro que SAST no lo cubre todo y que herramientas como DAST o las pruebas manuales siguen siendo necesarias para los problemas de lógica y de configuración que no se ven en el código.